

Reviewer Pack — Minimal Hodge Norm and Frege Proof Length Inequality

Entry 365eb9daf7bd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 22:32:56 UTC

Minimal Hodge Norm and Frege Proof Length Inequality

Entry ID: 365eb9daf7bd **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 22:32:56 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Frege Proof Complexity

Statement:

For any given Tseitin formula φ with n variables, the Frege proof length of φ is upper-bounded by the square root of the minimal Hodge norm of its associated algebraic variety $V(\varphi)$, i.e., $w_{\text{Frege}}(\varphi) \leq \sqrt{H_{\min}(V(\varphi))}$.

Rationale (proposer's reasoning):

The relationship between Hodge theory and complexity theory has not been extensively explored, but Hodge structures are known to be complex objects. This conjecture suggests that the algebraic complexity of a variety could provide insights into the proof length required for certain types of proofs, offering a potential new approach to understanding proof complexity.

Taxonomy category: HODGE_T0_FREGE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 931e4ff8473bdae0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a Tseitin formula φ with n variables, if the square root of the minimal Hodge norm $\sqrt{H_{\min}(V(\varphi))}$ is less than or equal to the Frege proof length $w_{\text{Frege}}(\varphi)$ for all seeds and their corresponding metrics, the conjecture is supported. Falsification occurs if any seed results in a Frege proof length greater than $\sqrt{H_{\min}(V(\varphi))}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def tseitin_formula(n):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []
        for i in range(1, n+1):
            clauses.append([variables[i-1]])
            clauses.append([-variables[i-1], random.choice(variables)])
        return clauses

    def frege_proof_length(clauses):
        # Simplified estimation of Frege proof length
        return len(clauses) * 2

    def hodge_norm(n):
        # Simplified estimation of Hodge norm
        return math.sqrt(n)

    n = random.choice([5, 10, 15, 20, 30, 40])
    formula = tseitin_formula(n)
    proof_length = frege_proof_length(formula)
    hodge_norm_value = hodge_norm(n)

    return {
        "metric_name": "Frege_proof_length_bound",
        "metric_value": proof_length,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": proof_length <= math.sqrt(hodge_norm_value),
        "counterexample": f"Frege proof length {proof_length} > Hodge norm bound {math.sqrt(hodge_norm_value)}"
    }

if __name__ == "__main__":
```

```

import sys
seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"Frege proof length > Hodge norm bound\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unsupported")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9af0a701.py", line 57, in <module>

```

    result = run_trial(seed)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9af0a701.py", line 38, in run_trial

```

    formula = tseitin_formula(n)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9af0a701.py", line 26, in tseitin_formula

```

    clauses.append([-variables[i-1], random.choice(variables)])
    ~~~~~

```

TypeError: bad operand type for unary -: 'str'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to provide a result for the conjecture. | next: Review and debug the test code to ensure it can run to completion without errors.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18279
2	preregistration	ollama_remote	glm4:latest	0	0	15884
3	novelty	ollama_remote	glm4:latest	0	0	20349
4	novelty	ollama_remote	glm4:latest	0	0	9182
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12162
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8365
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13983
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7548
9	judge	ollama_remote	glm4:latest	0	0	8628

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 114379 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/365eb9daf7bd.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/365eb9daf7bd.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/365eb9daf7bd.tar.gz` (if generated)